WESTCLIFF
UNIVERSITY
Educate. Inspire. Empower.

REACT

Review

Module 5 - Week 1 Day 3

Agenda

- What is React, why use it?
- Setting up React
- What are Components, why and how to create
- What is JSX, purpose, how to create
- What is Babel, purpose
- What is Reactstrap, how to use
- Using Conditions in React
- How to use Style, Class, Comment, etc.
- What is defaultProps
- What and why Functional Components
- What is State, State vs Props
- What are methods and events
- Creating infinite loops
- Mount and unmount components
- Updating components
- Handling errors
- Lab Assignment

What is React, Why use it?

What is React?

React is a JavaScript library for building user interfaces. It is maintained by Facebook and a community of individual developers and companies. It was first released in **May 2013** by the original author **Jordan Walke**.

The current version is **17.0.2**.

React has been designed from the start for gradual adoption, and you can use as little or as much React as you need. Whether you want to get a taste of React, add some interactivity to a simple HTML page, or start a complex React-powered app.

React is Declarative, Component-Based and can be used anywhere.

What is React?

Declarative: React makes it painless to create interactive UIs. Design simple views for each state in your application, and React will efficiently update and render just the right components when your data changes.

Declarative views make your code **more predictable** and **easier to debug**.

Component-Based: Build **encapsulated components** that manage their own state, then compose them to make complex UIs.

It can be used anywhere: React can also render on the server using Node and power mobile apps using React Native. It can work with any stack.

What is React?

React DOM (Virtual Document Object Model)

React creates an in-memory data structure cache which computes the changes made and then updates the browser. This allows a special feature that enables the programmer to code as if the whole page is rendered on each change whereas react library only renders components that actually change.

Why use React?

React.js is currently the most popular front-end JavaScript library for building Web applications. React.js or Reactjs or simply React are different ways to represent React.js. Most fortune 500 companies use Reactjs.

React is different from other JS frameworks.

Why use React?

Some React Features:

- React is declarative
- React is simple
- React is component-based
- React supports server-side
- React supports mobile development
- React is extensible
- React is fast
- React is easy to learn

Why use React?

1. **Simplicity**: ReactJS is just simpler to grasp right away. The component-based approach, well-defined lifecycle, and use of just plain JavaScript make React very simple to learn, build a professional web or a mobile applications, and support it.

2. **Easy to learn**: Anyone with a basic previous knowledge in programming can easily understand React.To react, you just need basic knowledge of CSS and HTML.

3. **Native Approach** (React Native): React can be used to create mobile applications and extensive code reusability is supported. So at the same time, we can make IOS, Android and Web applications.

Why use React?

4. **Data Binding**: React uses one-way data binding and an application architecture called Flux controls the flow of data to components through one control point – the dispatcher. It's easier to debug self-contained components of large ReactJS apps.

5. **Performance**: React does not offer any concept of a built-in container for dependency. You can use Babel, ReactJS-di to inject dependencies automatically.

6. **Testability:** ReactJS applications are super easy to test. React views can be treated as functions of the state, so we can manipulate with the state we pass to the ReactJS view and take a look at the output and triggered actions, events, functions, etc.

Setting up React

Setting up React?

There are few ways to use React. **Method 1**:

CDN

For applications that do not use NPM as its server setup, we can use the hosted CDN for development and production use. For development use: React and React DOM links:

```
  <!-- ... other HTML ... -->

  <!-- Load React. -->
  <!-- Note: when deploying, replace "development.js" with
"production.min.js". -->
  <script src="https://unpkg.com/react@17/umd/react.development.js"
crossorigin></script>
  <script src="https://unpkg.com/react-dom@17/umd/react-
dom.development.js" crossorigin></script>

  <!-- Load our React component. -->
  <script src="like_button.js"></script>

</body>
```

Setting up React?

Method 2:

Using NPM

This package serves as the entry point to the DOM and server renderers for React. It is intended to be paired with the generic React package, which is shipped as react to npm. You have to install **React** and **React DOM**:

```
npm install react react-dom
```

….and to use them in your component:

```
const React = require('react’);
const ReactDOMServer = require('react-dom');
```

Setting up React?

```
<script>
   class HelloMessage extends React.Component {
       render() {
       return React.createElement(
           "div",
           null,
           "Hello ",
           this.props.name
       );
       }
   }
    ReactDOM.render(React.createElement(HelloMessage,
       { name: "Westcliff" }),
       document.getElementById('hello-example'));
</script>
```

Class component extending React

Render method that takes input data and returns what to display.

Input data that is passed into the component can be accessed by render() via this.props

What are Components why and how to create

What are components?

React lets you define components as classes or functions. Components defined as classes currently provide more features which are described in detail on this page. All components are created to be reusable. To define a React component class, you need to extend React.Component:

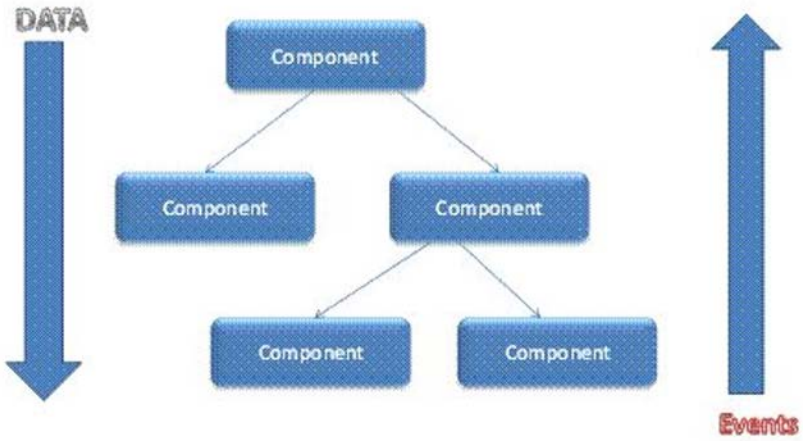
```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

The only method you must define in a **React.Component** subclass is called **render()**. All the other methods described on this page are optional.

What are components?

In the previous example we saw how to create a simple component.

In React, a set of immutable values are passed to the components renderer as properties (props) in its HTML tags. The component cannot directly modify any properties but can pass a callback function with the help of which we can do modifications. This complete process is known as “properties flow down;
actions flow up”.



What are components?

There are stateful components that set a state.

In this example the tick function is setting a state. Once the state is set, it can be updated.

```
tick() {
 this.setState(state => ({
   seconds: state.seconds + 1
 }));
}
```

setState() schedules an update to a component's state object. When state changes, the component responds by re-rendering.

Props and **State** are both plain JavaScript objects. While both hold information that influences the output of render, they are different in one important way: props get passed to the component (similar to function parameters) whereas state is managed within the component

Components

The Component Lifecycle

Each component has several “lifecycle methods” that you can override to run code at particular times in the process.

1) Mounting

These methods are called in the following order when an instance of a component is being created and inserted into the DOM:

- constructor()
- static getDerivedStateFromProps()
- render()
- componentDidMount()

Components

2) Updating

An update can be caused by changes to props or state. These methods are called in the following order when a component is being re-rendered:

- static getDerivedStateFromProps()
- shouldComponentUpdate()
- render()
- getSnapshotBeforeUpdate()
- componentDidUpdate()

Components

3) Unmounting

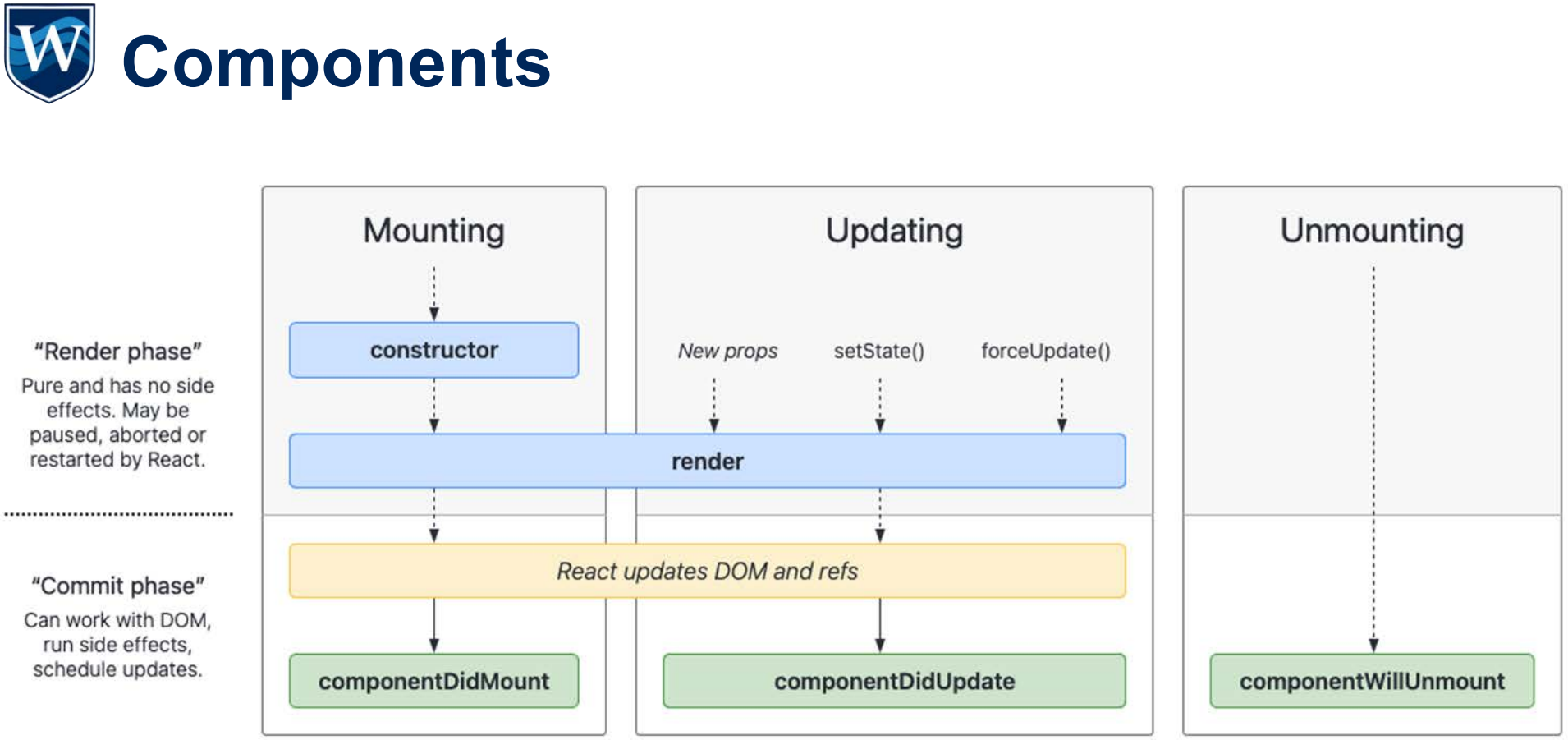
This method is called when a component is being removed from the DOM:

- componentWillUnmount()

4) Error Handling

These methods are called when there is an error during rendering, in a lifecycle method, or in the constructor of any child component.

- static getDerivedStateFromError()
- componentDidCatch()



Components

"Render phase"
Pure and has no side effects. May be paused, aborted or restarted by React.
"Commit phase"
Can work with DOM, run side effects, schedule updates.
Mounting
Updating
Unmounting
constructor
New props
setState()
forceUpdate()
render
React updates DOM and refs
componentDidMount
componentDidUpdate
componentWillUnmount

Components

The **render()** method is the only required method in a class component.

It examine **this.props** and **this.state** and return one of the following types:

- React elements. Typically created via JSX. For example, and <MyComponent /> are React elements that instruct React to render a DOM node
- Arrays and fragments. Let you return multiple elements from render.
- Portals. Let you render children into a different DOM subtree.
- String and numbers. These are rendered as text nodes in the DOM.
- Booleans or null. Render nothing. (Mostly exists to support return test && <Child /> pattern, where test is boolean.)

The **render()** function should be pure, meaning that it does not modify component state, it returns the same result each time it's invoked, and it does not directly interact with the browser.

Components

The **constructor** for a React component is called before it is mounted. When implementing the constructor for a React.Component subclass, you should call **super(props)** before any other statement. Otherwise, **this.props** will be undefined in the constructor, which can lead to bugs.

Typically, in React constructors are only used for two purposes:

- Initializing local state by assigning an object to this.state.
- Binding event handler methods to an instance.

You should not call **setState()** in the **constructor()**. Instead, if your component needs to use local state, assign the initial state to this.state directly in the constructor

Components

```
constructor(props) {
  super(props);
  // Don't call this.setState() here!
  this.state = { counter: 0 };
  this.handleClick = this.handleClick.bind(this);
}
```

Constructor is the only place where you should assign this.state directly. In all other methods, you need to use **this.setState()** instead.

Avoid introducing any side-effects or subscriptions in the constructor. For those use cases, use **componentDidMount()** instead.

Components

componentDidMount() is invoked immediately after a component is mounted. Initialization that requires DOM nodes should go here. If you need to load data from a remote endpoint, this is a good place to instantiate the network request.

This method is a good place to set up any subscriptions. If you do that, don't forget to unsubscribe in **componentWillUnmount()**.

You may call **setState()** immediately in **componentDidMount()**. It will trigger an extra rendering, but it will happen before the browser updates the screen. This guarantees that even though the render() will be called twice in this case, the user won't see the intermediate state.

Components

componentDidUpdate() is invoked immediately after updating occurs. This method is not called for the initial render.

Use this as an opportunity to operate on the DOM when the component has been updated. This is also a good place to do network requests as long as you compare the current props to previous props.

You may call **setState()** immediately in **componentDidUpdate()** but note that it must be wrapped in a condition

```
componentDidUpdate(prevProps) {
  // Typical usage (don't forget to compare props):
  if (this.props.userID !== prevProps.userID) {
    this.fetchData(this.props.userID);
  }
}
```

Components

componentWillUnmount() is invoked immediately before a component is unmounted and destroyed. Perform any necessary cleanup in this method, such as invalidating timers, canceling network requests, or cleaning up any subscriptions that were created in **componentDidMount()**.

You should **not** call **setState()** in **componentWillUnmount()** because the component will never be re-rendered. Once a component instance is unmounted, it will never be mounted again.

What is JSX, purpose, how to create

JSX

JSX is a syntax extension to JavaScript. We recommend using it with React to describe what the UI should look like. JSX may remind you of a template language, but it comes with the full power of JavaScript. JSX produces React “elements”. We will explore rendering them to the DOM.

```
const element = <h1>Hello, world!</h1>;  // JSX tag
```

Why JSX?

React embraces the fact that rendering logic is inherently coupled with other UI logic:

- how events are handled
- how the state changes over time
- how the data is prepared for display

Instead of artificially separating technologies by putting markup and logic in separate files, React separates concerns with loosely coupled units called “components” that contain both.

React doesn’t require using JSX, but most developers find it helpful as a visual aid when working with UI inside the JavaScript code. It also allows React to show more useful error and warning messages.

Why JSX?

Embedding Expressions in JSX

we declare a variable called **name** and then use it inside JSX by wrapping it in curly braces:

```
const name = 'West Cliff;
const element = <h1>Hello, {name}</h1>;
ReactDOM.render(
    element,
    document.getElementById('root')
);
```

You can put any valid JavaScript expression inside the curly braces in JSX.

Why JSX?

we embed the result of calling a JavaScript function, **formatName(user)**, into an **<h1>** element.

```
function formatName(user) {
  return user.firstName + ' ' + user.lastName;
}

const user = {
  firstName: 'West',
  lastName: 'Cliff'
};

const element = (
  <h1>
    Hello, {formatName(user)}!  </h1>
);

ReactDOM.render(
  element,
  document.getElementById('root')
);
```

Why JSX?

Expressions in JSX

JSX expressions become regular JavaScript function calls and evaluate to JavaScript objects. This means that you can use JSX inside of if statements and for loops, assign it to variables, accept it as arguments, and return it from functions

```
function getGreeting(user) {
  if (user) {
    return <h1>Hello, {formatName(user)}!</h1>;  }
  return <h1>Hello, Stranger.</h1>;}
```

Why JSX?

Attributes in JSX

You may use quotes to specify string literals as attributes:

```
const element = <div tabIndex="0"></div>;
```

You may also use curly braces to embed a JavaScript expression in an attribute:

```
const element = <img src={user.avatarUrl}></img>;
```

Don’t put quotes around curly braces when embedding a JavaScript expression in an attribute. You should either use quotes or curly braces, but not both in the same attribute.

Why JSX?

Specifying Children with JSX

If a tag is empty, you may close it immediately with />, like XML:

```
const element = <img src={user.avatarUrl} />;
```

JSX tags may contain children:

```
const element = (
  <div>
    <h1>Hello!</h1>
    <h2>Good to see you here.</h2>
  </div>
);
```

Why JSX?

JSX Prevents Injection Attacks

It is safe to embed user input in JSX:

```
const title = response.potentiallyMaliciousInput;
// This is safe
const element = <h1>{title}</h1>;
```

By default, React DOM escapes any values embedded in JSX before rendering them. Thus it ensures that you can never inject anything that's not explicitly written in your application. Everything is converted to a string before being rendered. This helps prevent ***XSS (cross-site-scripting)*** attacks.

Why JSX?

JSX Represents Objects

Babel compile JSX down to **React.createElement()** calls.

These two examples are identical:

```
const element = (
  <h1
className="greeting">
    Hello, world!
  </h1>
);
```

```
const element = React.createElement (
  'h1',
  {className: 'greeting'},
  'Hello, world!'
);
```

Why JSX?

React.createElement() performs a few checks to help you write bug-free code but essentially it creates an object like this:

```
// Note: this structure is simplified
const element = {
  type: 'h1',
  props: {
    className: 'greeting',
    children: 'Hello, world!'
  }
};
```

These objects are called “React elements”. You can think of them as descriptions of what you want to see on the screen.

React reads these objects and uses them to construct the DOM and keep it up to date.

What is Babel, purpose

What is Babel?

Babel is a JavaScript compiler

Babel is a toolchain that is mainly used to convert ECMAScript 2015+ code into a backwards compatible version of JavaScript in current and older browsers or environments. Here are the main things Babel can do for you:

- Transform syntax
- Polyfill features that are missing in your target environment (through a third-party polyfill such as core-js)
- Source code transformations (codemods)

What is Babel?

```
// Babel Input: ES2015 arrow function
[1, 2, 3].map((n) => n + 1);

// Babel Output: ES5 equivalent
[1, 2, 3].map(function(n) {
  return n + 1;
});
```

Babel

JSX and React

Babel can convert JSX syntax! Use it together with the babel-sublime package to bring syntax highlighting to a whole new level.

You can install it with NPM

```
npm install --save-dev @babel/preset-react
```

Babel

Type Annotations (Flow and TypeScript)

Babel can strip out type annotations! Keep in mind that Babel doesn't do type checking; you'll still have to install and use Flow or TypeScript to check types.

You can install the flow preset and typescript with NPM

npm install --save-dev @babel/preset-flow

```
// @flow
function square(n: number): number {
  return n * n;
}
```

npm install --save-dev @babel/preset-typescript

```
function Greeter(greeting: string) {
    this.greeting = greeting;
}
```

What is Reactstrap, how to use

Reactstrap

This library contains React Bootstrap 4 components that favor composition and control. The library does not depend on jQuery or Bootstrap javascript. However, popper.js is relied upon for advanced positioning of content like Tooltips, Popovers, and auto-flipping Dropdowns.

Reactstrap

Install reactstrap and peer dependencies via NPM

```
npm install --save reactstrap react react-dom
```

OR

Install reactstrap and Bootstrap from NPM. Reactstrap does not include Bootstrap CSS so this needs to be installed seprately

```
npm install --save bootstrap
```

Import Bootstrap CSS in the src/index.js file:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Reactstrap

CDN

Reactstrap can be included directly in your application's bundle or excluded during compilation and linked directly to a CDN.

https://cdnjs.cloudflare.com/ajax/libs/reactstrap/4.8.0/reactstrap.min.js

Note: When using the external CDN library, be sure to include the required dependencies as necessary prior to the Reactstrap library.

Reactstrap

There are a few core concepts to understand in order to make the most out of this library.

1) Your content is expected to be composed via props.children rather than using named props to pass in Components.

```
// Content passed in via props
const Example = (props) => {
  return (
    <p>This is a tooltip <TooltipTrigger
tooltip={TooltipContent}>example</TooltipTrigger>!
</p>
  );
}
```

```
// Content passed in as children (Preferred)
const PreferredExample = (props) => {
  return (
    <p>
      This is a <a href="#" id="TooltipExample">tooltip</a>
example.
      <Tooltip target="TooltipExample">
        <TooltipContent/>
      </Tooltip>
    </p>
  );
}
```

Reactstrap

2) Attributes in this library are used to pass in state, conveniently apply modifier classes, enable advanced functionality (like popperjs), or automatically include non-content based elements.

- **isOpen** - current state for items like dropdown, popover, tooltip
- **toggle** - callback for toggling isOpen in the controlling component
- **color** - applies color classes, ex: <Button color="danger"/>
- **size** for controlling size classes. ex: <Button size="sm"/>
- **tag** - customize component output by passing in an element name or Component
- **boolean** based props (attributes) when possible for alternative style classes or sr-only content

Reactstrap

```
import React from 'react';
import { Button } from 'reactstrap';

export default (props) => {

  return (

    <Button color="Blue">Danger!</Button>

  );

};
```

First import React

Then import Button from reactstrap so we can use it like Bootstrap

We use the button with color danger or other bootstrap colors

Using Conditions in React

Conditional Rendering

Conditional rendering in React works the same way conditions work in JavaScript. Use JavaScript operators like if or the conditional operator to create elements representing the current state, and let React update the UI to match them.

Consider these two components:

```
function UserGreeting(props) {
    return <h1>Welcome back!</h1>;
}

function GuestGreeting(props) {
     return <h1>Please sign up.</h1>;
}
```

Conditional Rendering

We'll create a Greeting component that displays either of these components depending on whether a user is logged in:

```
function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
      return <UserGreeting />;
   }
 return <GuestGreeting />;
}
ReactDOM.render(
 // Try changing to isLoggedIn={true}:
<Greeting isLoggedIn={false} />,
document.getElementById('root'));
```

If loggedin, calls a function

If not loggedin, calls a function

Conditional Rendering

Element Variables

You can use variables to store elements. This can help you conditionally render a part of the component while the rest of the output doesn't change.

```
class LoginControl extends React.Component {
  constructor(props) {
    super(props);
    this.handleLoginClick = this.handleLoginClick.bind(this);
    this.handleLogoutClick = this.handleLogoutClick.bind(this);
    this.state = {isLoggedIn: false};
  }

  handleLoginClick() {
    this.setState({isLoggedIn: true});
  }

  handleLogoutClick() {
    this.setState({isLoggedIn: false});
  }
```

Conditional Rendering

We will create a stateful component called *LoginControl*. It will render either **<LoginButton />** or **<LogoutButton />** depending on its current state. It will also render a **<Greeting />**

While declaring a variable and using an if statement is a fine way to conditionally render a component, sometimes you might want to use a shorter syntax.

```
render() {
  const isLoggedIn = this.state.isLoggedIn;
  let button;
  if (isLoggedIn) {
    button = <LogoutButton onClick={this.handleLogoutClick} />;
  } else {
    button = <LoginButton onClick={this.handleLoginClick} />;
  }

  return (
    <div>
      <Greeting isLoggedIn={isLoggedIn} />
      {button}
    </div>
  );
}
```

Conditional Rendering

You may embed expressions in JSX by wrapping them in curly braces. This includes the JavaScript logical && operator. It can be handy for conditionally including an element

In JavaScript, true && expression always evaluates to true expression, and false && expression always evaluates to false.

Therefore, if the condition is true, the element right after && will appear in the output. If it is false, React will ignore and skip it.

```
function Mailbox(props) {
  const unreadMessages = props.unreadMessages;
  return (
    <div>
      <h1>Hello!</h1>
      {unreadMessages.length > 0 &&
        <h2>
          You have {unreadMessages.length} unread messages.
        </h2>
      }
    </div>
  );
}
```

Conditional Rendering

Inline If-Else with Conditional Operator

Another method for conditionally rendering elements inline is to use the JavaScript conditional operator condition ? true : false

```
render() {
  const isLoggedIn = this.state.isLoggedIn;
  return (
    <div>
      The user is <b>{isLoggedIn ? 'currently' : 'not'}</b> logged
in.
    </div>
  );
}
```

How to use Style, Class, Comment, etc.

Styling REACT

React has a special markup language called JSX. It is a mix of HTML and JavaScript syntax that looks something like this:

```
<div>
    {inventory.filter(item => item.available).map(item => (
   <Card>
      <div className="title"}>{item.name}</div>
      <div className="price">{item.price}</div>
   </Card>
  ))
}
</div>
```

We have seen .filter and .map, and HTML like . Other parts that look like both HTML and JavaScript, such as <Card> and className.

This is JSX, the special markup language that gives React components the feel of HTML with the power of JavaScript.

Styling REACT

How to Add CSS classes to components?

Pass a string as the className prop:

```
render() {
  return <span className="menu navigation-menu">Menu</span>
}
```

Styling REACT

Can I use inline styles?

Yes

Are inline styles bad?

CSS classes are generally better for performance than inline styles.

What is CSS-in-JS?

“CSS-in-JS” refers to a pattern where CSS is composed using JavaScript instead of defined in external files.

Styling REACT

There are many ways to style React with CSS, this tutorial will take a closer look at inline styling, and CSS stylesheet.

Inline Styling

To style an element with the inline style attribute, the value must be a JavaScript object

```
render() {
   return (
     <div>
     <h1 style={{color: "red"}}>Hello Style!</h1>
     <p>Add a little style!</p>
     </div>
   );
 }
```

Styling REACT

camelCased Property Names

Since the inline CSS is written in a JavaScript object, properties with two names, like background-color, must be written with camel case syntax

Use **backgroundColor** instead of *background-color*:

```
render() {
   return (
     <div>
     <h1 style={{backgroundColor: "lightblue"}}>Hello Style!</h1>
     <p>Add a little style!</p>
     </div>
   );
 }
```

Styling REACT

JavaScript Object

You can also create an object with styling information, and refer to it in the style attribute

```
class MyHeader extends React.Component {
  render() {
    const mystyle = {
      color: "white",
      backgroundColor: "Blue",
      padding: "15px",
      fontFamily: "Arial"
    };
    return (
      <div>
      <h1 style={mystyle}>Hello Style!</h1>
      <p>Add a little style!</p>
      </div>
    );
  }
}
```

Styling REACT

You can write your CSS styling in a separate file, just save the file with the .css file extension, and import it in your application.

App.css:

Create a new file called "App.css" and insert some CSS code in it:

```
body {
    background-color: #282c34;
    color: white;
    padding: 40px;
    font-family: Arial;
    text-align: center;
}
```

Add `import './App.css';`
- to import the .css file

What is defaultProps

What is defaultProps?

Components are the main tools we use to build an interactive UI with. All React applications have a root component, often called the App component.

React provides ways for components to pass data to each other and to respond to each other's events.

You can write a component and use it as a child component in several other components — they were designed to be self-contained and loosely coupled for this purpose.

Each component contains valuable data about itself:

- What data it needs as input
- How to draw itself

What is defaultProps?

A normal props with React components take inputs in the props argument.

```
// ES6 class
class CatComponent extends React.Component {
   constructor(props) {
   render() {
      return <div>{this.props.catName} Cat</div>
   }
}
// Functional component
function CatComponent(props) {
   return <div>{props.catName} Cat</div>
}
```

The problem: what if the parent component doesn't pass any attributes to the child component.

You will get **undefined**

What is defaultProps?

We can use the logical operator **||** to set a fallback value, so when a prop is missing it displays the fallback value in place of the missing prop.

```
// ES6 class
class CatComponent extends React.Component {
   constructor(props) {}
   render() {
      return <div>{this.props.catName || "Sandy"} Cat, Eye Color: {this.props.eyeColor ||
 "deepblue" }, Age: {this.props.age || "120"}</div>
   }
}
// Functional component
function CatComponent(props) {
   return <div>{props.catName || "Sandy"} Cat, Eye Color: {props.eyeColor || "deepblue"},
 Age: {props.age || "120"}</div>
}
```

If no value for catName is passed. || will be default

If no value for catName is passed. || will be default

What is defaultProps?

Now, previous example works well but you do not want to go through and attach || to all of your code. In this case we use **defaultProps**.

defaultProps is a property in React component used to set default values for the props argument. It will be changed if the prop property is passed.

defaultProps can be defined as a property on the component class itself, to set the default props for the class.

What is defaultProps?

we will define a static property named **defaultProps**.

```
class ReactComp extends React.Component {}
ReactComp.defaultProps = {}

// or
class ReactComp extends React.Component {
  static defaultProps = {}
}
```

What is defaultProps?

Using the previous cat example:

```
// ES6 class
class CatComponent extends React.Component {
   constructor(props) {}
   render() {
       return
            <div>
                {this.props.catName} Cat, Eye Color: {this.props.eyeColor },
                Age: {this.props.age}
            </div>
   }
}

CatComponent.defaultProps = {
   catName: "Sandy",
   eyeColor: "deepblue",
   age: "120"
}
```

What is defaultProps?

Or something like this:

```
class CatComponent extends React.Component {
   constructor(props) {}
   static defaultProps = {
      catName: "Sandy",
      eyeColor: "deepblue",
      age: "120"
   }
   render() {
      return
         <div>
            {this.props.catName} Cat, Eye Color: {this.props.eyeColor }, Age:
            {this.props.age}
         </div>
   }
}
```

What and why Functional Components

Functional Components

In React we can use functions as components to render views. As with its ES6 component counterpart, we can add default props to the component by adding the static property defaultProps to the function:

```
function Reactcomp(props) {}

ReactComp.defaultProps = {}
```

Functional Components

Using our cat example:

Without a Class

```
// Functional component
function CatComponent(props) {
   return <div>{props.catName} Cat, Eye Color: {props.eyeColor}, Age: {props.age}</div>
}
CatComponent.defaultProps = {
   catName: "Sandy",
   eyeColor: "deepblue",
   age: "120"
}
```

Functional Components

Functional component explanation:

1. We call ReactDOM.render() with the <Welcome name="Sara" /> element.
2. React calls the Welcome component with {name: 'Sara'} as the props.
3. Our Welcome component returns a <h1>Hello, Sara</h1> element as the result.
4. React DOM efficiently updates the DOM to match <h1>Hello, Sara</h1>.

```
function Welcome(props) {
      return <h1>Hello, {props.name}</h1>;
}
const element = <Welcome name="Sara" />;
ReactDOM.render(
  element,
  document.getElementById('root')
);
```

What is State, State vs Props

React State

React components has a built-in state object.

The state object is where you store property values that belongs to the component.

When the state object changes, the component re-renders.

Creating a State:

The state object is
initialized in the constructor
as discussed in day 1 lesson.

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = {brand: "Ford"};
  }
  render() {
    return (
      <div>
        <h1>My Car</h1>
      </div>
    );
  }
}
```

React State

The state object can contain as many properties as you like.

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      brand: "Ford",
      model: "Mustang",
      color: "red",
      year: 1964
    };
  }
```

React State

Using the State Object

State object anywhere in the component by using the this.state.propertyname

Refer to the state object in the render() method:

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      brand: "Ford",
      model: "Mustang",
      color: "red",
      year: 1964
    };
  }
  render() {
    return (
      <div>
        <h1>My {this.state.brand}</h1>
        <p> It is a {this.state.color}
          {this.state.model} from {this.state.year}.</p>
      </div>
    );
  }
}
```

React State

Changing the State Object

To change a value in the state object, use the this.setState() method.

When a value in the state object changes, the component will re-render

Always use the **setState()** method to change the state object, it will ensure that the component knows its been updated and calls the **render()** method

From previous example:

```
changeColor = () => {
   this.setState({color: "blue"});
 }
 render() {
   return (
     <div>
       <h1>My {this.state.brand}</h1>
       <p>It is a {this.state.color}
         {this.state.model}
         from {this.state.year}.</p>
       <button
         type="button"
         onClick={this.changeColor}
       >Change color</button>
     </div>
   );
 }
}
```

What are methods and events

React Methods

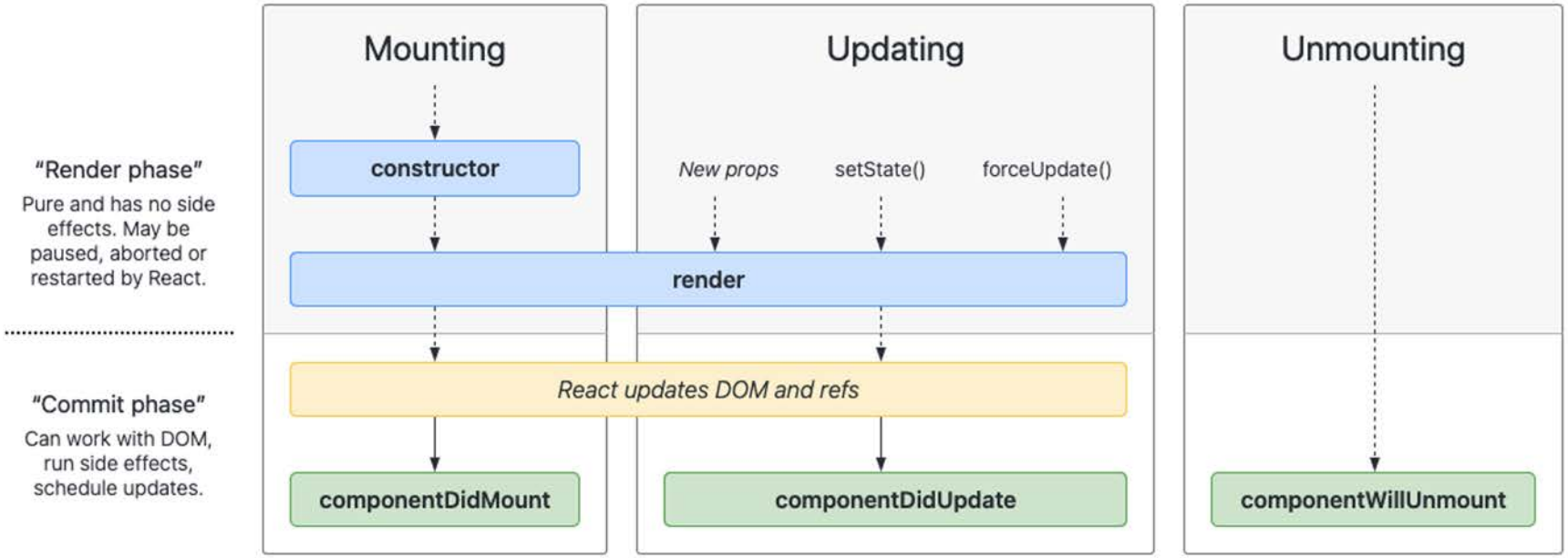
React lets you define components as classes or functions. Components defined as classes currently provide more features which are described in detail on this page. All components are created to be reusable. To define a React component class, you need to extend React.Component:

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

The only method you must define in a **React.Component** subclass is called **render()**. All the other methods described on this page are optional.

React Methods

Life cycle methods



React Events

React has the same events as HTML: click, change, mouseover etc.

Adding Events

React events are written in camelCase syntax:

onClick instead of onclick.

React event handlers are written inside curly braces:

onClick={clickbutton} instead of onClick="clickbutton()"

HTML

```
<button onclick="shoot()">Take the Shot!</button>
```

React Events

Event Handlers

Use the event handler as a method in the component class

Put the shoot function inside the Football component

```
class Football extends React.Component {
  shoot() {
    alert("Great Shot!");
  }
  render() {
    return (
      <button onClick={this.shoot}>Take the
shot!</button>
    );
  }
}

ReactDOM.render(<Football />,
document.getElementById('root'));
```

React Events

Bind this

For methods in React, the this keyword should represent the component that owns the method.

Always use arrow functions, this will always represent the object that defined the arrow function.

```
class Football extends React.Component {
  shoot = () => {
    alert(this);
    /*The 'this' keyword refers to the component object */
  }
  render() {
    return (
      <button onClick={this.shoot}>Take the
shot!</button>
    );
  }
}

ReactDOM.render(<Football />,
document.getElementById('root'));
```

React Events

Arrow Functions

In class components, the this keyword is not defined by default, so with regular functions the this keyword represents the object that called the method, which can be the global window object or a HTML button.

If you must use regular functions instead of arrow functions you have to bind this to the component instance using the bind() method

```
class Football extends React.Component {
  constructor(props) {
    super(props)
    this.shoot = this.shoot.bind(this)
  }
  shoot() {
    alert(this);
    /* Thanks to the binding in the constructor
function,  the 'this' keyword now refers to the
component object */
  }
```

React Events

Passing Arguments

If you want to send parameters into an event handler, you have two options:

1. Make an anonymous arrow function:

```
class Football extends React.Component {
  shoot = (a) => {
    alert(a);
  }
  render() {
    return (
      <button onClick={() => this.shoot("Goal")}>Take the
shot!</button>
    );
  }
}
```

React Events

Passing Arguments

2. Bind the event handler to *this*. The first argument has to be *this*.

```
class Football extends React.Component {
  shoot(a) {
    alert(a);
  }
  render() {
    return (
      <button onClick={this.shoot.bind(this, "Goal")}>Take
the shot!</button>
    );
  }
}
```

Creating infinite loops

Infinite loops

Infinite loop happens when you use a loop and do not clean up your code in React DOM.

useEffect() - (also known as cleanup) React hook manages the side-effects like fetching over the network, manipulating DOM directly, starting and ending timers.

While the useEffect() is, alongside with useState(), is one of the most used hooks, it requires some time to familiarize and use correctly.

Infinite loops

A functional component contains an input element. In this example the code will count and display how many times the input has changed.

In this example:

<input type="text" value={value} onChange={onChange} /> is a controlled component. value state variable holds the input value, and the onChange event handler updates the value state when user types into the input.

```
function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);

  useEffect(() => setCount(count + 1));
  const onChange = ({ target }) => setValue(target.value);

  return (
    <div>
      <input type="text" value={value} onChange={onChange} />
      <div>Number of changes: {count}</div>
    </div>
  )
}
```

Infinite loops

A functional component contains an

Every time the component re-renders due to user typing into the input, the useEffect(() => setCount(count + 1)) updates the counter.

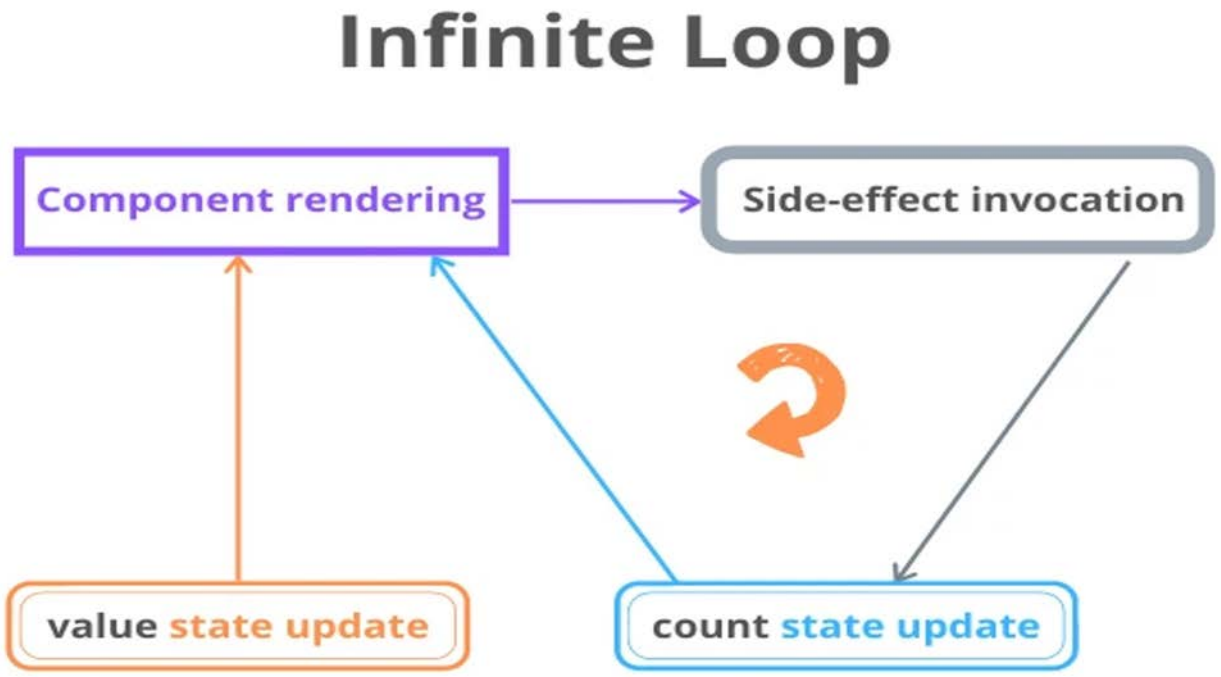
useEffect(() => setCount(count + 1)) is used without dependencies argument, () => setCount(count + 1) callback is executed after every rendering of the component.

```
function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);

  useEffect(() => setCount(count + 1));
  const onChange = ({ target }) => setValue(target.value);

  return (
    <div>
      <input type="text" value={value} onChange={onChange} />
      <div>Number of changes: {count}</div>
    </div>
  )
}
```

Infinite loops



Mount and unmount components

Mount and unMount

The Component Lifecycle

Each component has several “lifecycle methods” that you can override to run code at particular times in the process.

Mounting

These methods are called in the following order when an instance of a component is being created and inserted into the DOM:

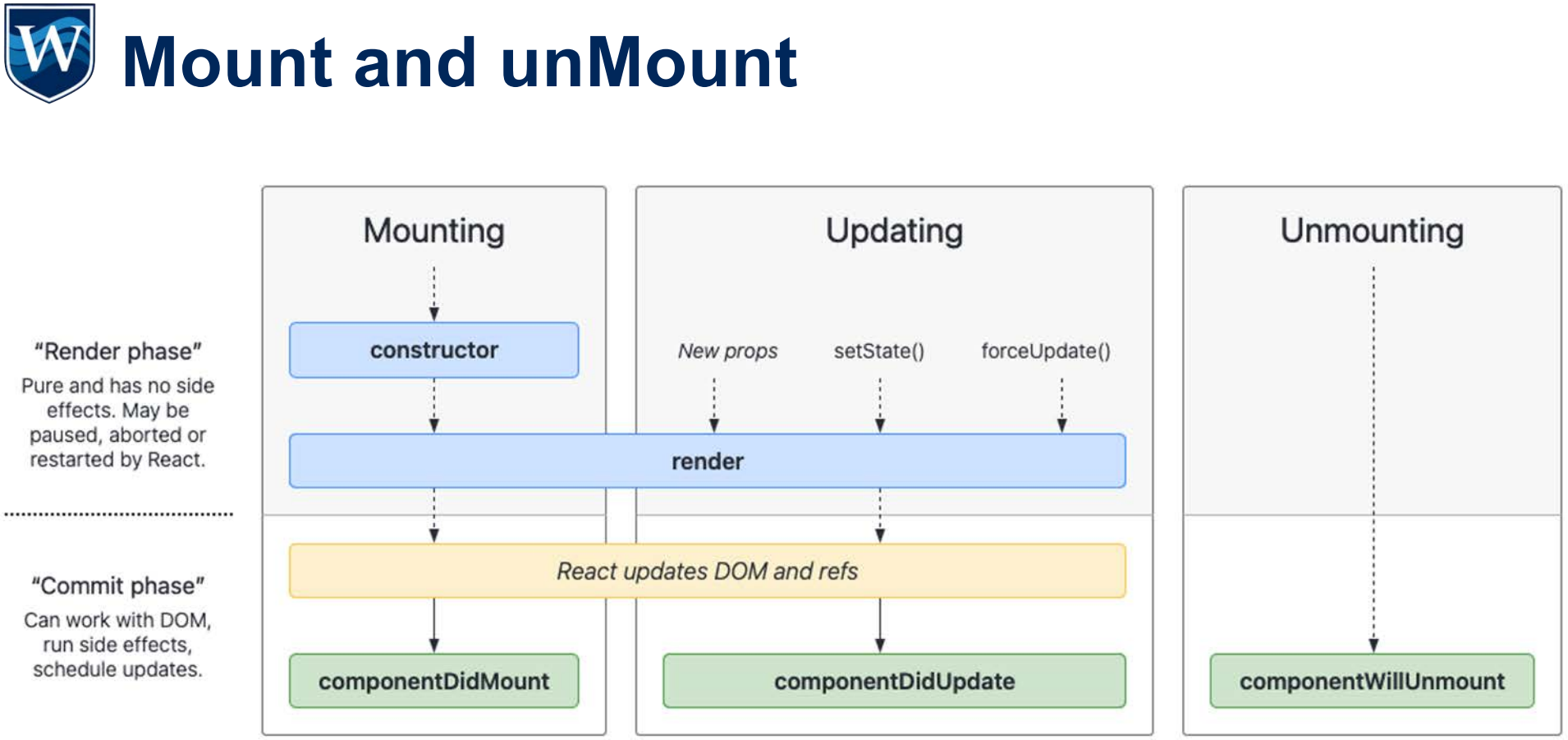
- constructor()
- static getDerivedStateFromProps()
- render()
- componentDidMount()

Mount and unMount

Unmounting

This method is called when a component is being removed from the DOM:

- componentWillUnmount()



Mount and unMount

"Render phase"

Pure and has no side effects. May be paused, aborted or restarted by React.

"Commit phase"

Can work with DOM, run side effects, schedule updates.

Mounting

constructor

render

React updates DOM and refs

componentDidMount

Updating

New props

setState()

forceUpdate()

render

React updates DOM and refs

componentDidUpdate

Unmounting

componentWillUnmount

Mount and unMount

componentDidMount() is invoked immediately after a component is mounted. Initialization that requires DOM nodes should go here. If you need to load data from a remote endpoint, this is a good place to instantiate the network request.

This method is a good place to set up any subscriptions. If you do that, don't forget to unsubscribe in **componentWillUnmount()**.

You may call **setState()** immediately in **componentDidMount()**. It will trigger an extra rendering, but it will happen before the browser updates the screen. This guarantees that even though the render() will be called twice in this case, the user won't see the intermediate state.

Mount and unMount

componentWillUnmount() is invoked immediately before a component is unmounted and destroyed. Perform any necessary cleanup in this method, such as invalidating timers, canceling network requests, or cleaning up any subscriptions that were created in **componentDidMount()**.

You should **not** call **setState()** in **componentWillUnmount()** because the component will never be re-rendered. Once a component instance is unmounted, it will never be mounted again.

Updating components

Updating Component

Updating

An update can be caused by changes to props or state. These methods are called in the following order when a component is being re-rendered:

- static getDerivedStateFromProps()
- shouldComponentUpdate()
- render()
- getSnapshotBeforeUpdate()
- componentDidUpdate()

Updating Component

componentDidUpdate() is invoked immediately after updating occurs. This method is not called for the initial render.

Use this as an opportunity to operate on the DOM when the component has been updated. This is also a good place to do network requests as long as you compare the current props to previous props.

You may call **setState()** immediately in **componentDidUpdate()** but note that it must be wrapped in a condition

```
componentDidUpdate(prevProps) {
  // Typical usage (don't forget to compare props):
  if (this.props.userID !== prevProps.userID) {
    this.fetchData(this.props.userID);
  }
}
```

Handling errors

Handling Errors

Error Handling

These methods are called when there is an error during rendering, in a lifecycle method, or in the constructor of any child component.

- static getDerivedStateFromError()
- componentDidCatch()

Handling Errors

Error Handling

When working with a component, when any error happens inside this component code React will unmount the whole React component tree, rendering nothing. This is the React way of handling crashes.

You don’t want errors to show up to your users. React decides to show a blank page.

However, this is just the default. Having a blank page show up is only slightly better than showing cryptic messages to users, but you should have a better way.

If you are in development mode, any error will trigger a detailed stack trace printed to the browser DevTools console. In production, where you don’t really want bugs printed out to your users.

Handling Errors

An error boundary is a React component that implements the componentDidCatch() lifecycle event, and wraps other components:

```
class ErrorHandler extends React.Component {
  constructor(props) {
    super(props)
    this.state = { errorOccurred: false }
  }
  componentDidCatch(error, info) {
    this.setState({ errorOccurred: true })
    logErrorToMyService(error, info)
  }
 render() {
    return this.state.errorOccurred ? <h1>Something went
wrong!</h1> : this.props.children
  }
}
```

Handling Errors

in a component JSX, you use it like this:

```
<ErrorHandler>
 <SomeOtherComponent />
</ErrorHandler>
```

```
class ErrorHandler extends React.Component {
  constructor(props) {
    super(props)
    this.state = { errorOccurred: false }
  }

  componentDidCatch(error, info) {
    this.setState({ errorOccurred: true })
    logErrorToMyService(error, info)
  }

  render() {
    return this.state.errorOccurred ? <h1>Something
went wrong!</h1> : this.props.children
  }
}
```

Resources 1

https://reactjs.org/

https://reactjs.org/docs/create-a-new-react-app.html#create-react-app

https://reactjs.org/docs/introducing-jsx.html

https://babeljs.io/repl/#?browsers=defaults%2C%20not%20ie%2011%2C%20not%20ie_mob%2011&build=&builtIns=false&spec=false&loose=false&code_lz=GYVwdgxgLglg9mABACwKYBt1wBQEpEDeAUIogE6pQhIIA8AJjAG4B8AEhlogO5xnr0AhLQD0jVgG4iAXyJA&debug=false&forceAllTransforms=false&shippedProposals=false&circleciRepo=&evaluate=false&fileSize=false&timeTravel=false&sourceType=module&lineWrap=true&presets=react&prettier=false&targets=&version=7.12.3&externalPlugins=

https://reactstrap.github.io/

https://www.w3schools.com/react/react_render.asp

https://javascript.info/import-export

https://reactstrap.github.io/components/card/

https://reactjs.org/docs/conditional-rendering.html

https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en

https://medium.com/@dan_abramov/smart-and-dumb-components-7ca2f9a7c7d0

Resources 2

https://blog.bitsrc.io/understanding-react-default-props-5c50401ed37d
https://www.w3schools.com/react/react_state.asp
https://reactjs.org/docs/react-component.html
https://www.w3schools.com/react/react_events.asp
https://flaviocopes.com/react-how-to-loop/
https://www.javatpoint.com/react-constructor
https://learn.co/lessons/react-component-mounting-and-unmounting
https://flaviocopes.com/react-handle-errors/
https://reactjs.org/docs/components-and-props.html
https://www.digitalocean.com/community/tutorials/react-class-components
https://www.w3schools.com/react/react_props.asp
https://reactjs.org/docs/lifting-state-up.html

Questions?